Experiment/Practical 12 Project /Case Study on Classification

1. **Title:**

Comparative Analysis of Naive Bayes, Support Vector Machine, Decision Tree and Random Forest for IoT Network Intrusion Detection Using the RT-IoT2022 Dataset

2. Aim:

To develop and compare four supervised machine-learning classification algorithms Naive Bayes, Support Vector Machine, Decision Tree, and Random Forest for detecting and classifying malicious network traffic in Internet of Things environments using the RT-IoT2022 dataset.

3. Problem Statement

- The rapid growth of IoT devices has increased the number of connected endpoints and consequently expanded the attack surface available to cybercriminals. Traditional intrusion-detection systems may struggle to identify diverse and evolving attacks in heterogeneous IoT networks.
- The RT-IoT2022 dataset contains real-time IoT network traffic along with multiple attack patterns such as DoS, DDoS, ARP poisoning, Nmap scanning, and SSH brute-force attacks. It contains 123,117 instances and 83 usable network-traffic features. UCI Machine Learning Repository
- The problem addressed in this study is: Can classical machine-learning classification algorithms accurately distinguish normal IoT network traffic from different types of cyberattacks, and which of Naive Bayes, SVM, Decision Tree and Random Forest provides the best balance between classification performance and computational efficiency?

4. **Literature Work** *(Minimum 5 recent papers from good quality journal):*

Sr. No.	Authors / Year	Paper	Dataset / Method	Major Findings
1	Sharmila & Nagapadma, 2023	Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset	RT-IoT2022; Quantized Autoencoder	Proposed a lightweight anomaly-detection approach for resource-constrained IoT devices and established RT-IoT2022 as a dataset for IoT intrusion-detection research. Springer Link
2	Airlangga, 2024	Comparative Analysis of Machine Learning Models for Intrusion Detection in Internet of Things Networks Using the RT-IoT2022 Dataset	RT-IoT2022; RF, Gradient Boosting, Logistic Regression, MLP	RF and Gradient Boosting achieved very high classification performance, demonstrating the usefulness of classical ML for IoT intrusion detection. IRPI
3	Sharmila et al., 2024	Performance Evaluation of Parametric and Non-Parametric Machine Learning Models using Statistical Analysis for RT-IoT2022 Dataset	RT-IoT2022; statistical analysis + ML	Used KS test, skewness, kurtosis, Pearson correlation and information gain. Decision Tree achieved 99.85% accuracy in their experimental setup. Open Research

4	2024	Enhancing IoT Network Security Using Feature Selection for Intrusion Detection Systems	RT-IoT2022; feature selection + ML	Investigated feature selection for reducing the dimensionality of RT-IoT2022 while retaining useful information for attack detection. MDPI
5	2025	TFKAN: Transformer based on Kolmogorov–Arnold Networks for Intrusion Detection in IoT environment	RT-IoT2022, IoT23, CICIoT2023	Demonstrated the continuing use of RT-IoT2022 as a benchmark for modern IoT intrusion-detection methods and compared advanced neural architectures across IoT datasets. ScienceDirect

Literature Review / Research Gap

Previous studies demonstrate that RT-IoT2022 is suitable for IoT intrusion detection. Existing research has investigated deep learning, feature selection, tree-based models, ensemble learning and advanced neural architectures.

However, there is still educational and practical value in conducting a controlled comparison of four fundamentally different classical classifiers Naive Bayes, SVM, Decision Tree and Random Forest using identical preprocessing, train-test splitting and evaluation criteria.

A particularly important issue is class imbalance. RT-IoT2022 contains highly unequal class frequencies, with some attack categories represented by only a few dozen samples while DOS_SYN_Hping contains tens of thousands of instances. UCI Machine Learning Repository

Therefore, this experiment will emphasize not only accuracy but also macro-averaged precision, recall, F1-score and confusion matrices.

5. **Objectives** *(Max 3)*

Objective 1

To preprocess and classify IoT network traffic into normal and malicious/attack categories using Naive Bayes, SVM, Decision Tree and Random Forest.

Objective 2

To compare the four algorithms using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix.

Objective 3

To identify the classifier that provides the best combination of classification performance, robustness to imbalanced classes and computational efficiency.

6. Methodology (*Proper steps and equation, also cover evaluation metrics required)*:

6.1 Overall Methodology

- The proposed methodology begins with the collection of the RT-IoT2022 dataset, followed by data understanding and exploratory data analysis (EDA) to examine the dataset structure, feature distributions, class distribution, and possible inconsistencies. The data is then cleaned by handling any irrelevant or unnecessary attributes, after which ID and other non-predictive attributes are removed to prevent them from influencing the classification models.
- Next, the categorical features are converted into numerical representations using appropriate encoding techniques. The processed dataset is then divided into training and testing sets using a stratified train-test split to maintain the distribution of attack classes. Feature scaling is subsequently performed where required, particularly for algorithms such as Naive Bayes and SVM.
- Four classification algorithms Naive Bayes, Support Vector Machine (SVM), Decision Tree, and Random Forest are then trained using the prepared training data. The trained models are used to predict the classes of the unseen test data. Their performance is evaluated using appropriate classification metrics, including accuracy, precision, recall, F1-score, macro-F1 score, confusion matrix, and computational time.
- Finally, the performance of all four classifiers is compared using numerical results and visualizations. Based on the evaluation metrics, the model that provides the most reliable classification performance, particularly for minority attack classes, is identified as the best-performing model for IoT intrusion detection.

6.2 Data Preprocessing

Let the dataset be represented as $D=\{(xi,yi)\}i=1N$, where xi represents the vector of network traffic features for the ith observation, yi represents its corresponding attack class, and N represents the total number of observations. The RT-IoT2022 dataset contains both numerical and categorical features and, according to the UCI Machine Learning Repository, does not contain missing values.

The first preprocessing step is to remove irrelevant columns, such as generated index or identification columns, because these attributes do not provide meaningful information for predicting the attack class and may negatively affect model performance. The target variable for the classification task is Attack_type, which represents the class of network traffic.

The second step is to encode categorical variables**.** Categorical network attributes, such as protocol or service-related features, cannot be directly processed by most machine-learning algorithms. Therefore, categorical features are converted into numerical representations using One-Hot Encoding**.**

The third step is feature scaling. Numerical features are standardized, particularly for algorithms such as SVM and Naive Bayes, using the standardization equation:

$z=x-\mu\sigma$

where x represents the original feature value, μ represents the mean of the feature, σ represents its standard deviation, and z represents the standardized feature value. Although Decision Tree and Random Forest algorithms do not require feature scaling, a consistent preprocessing pipeline can be used to maintain a systematic comparison between the models.

The final preprocessing step is to divide the dataset into training and testing **subsets**. In this study, 80% of the data is used for training the classification models, while the remaining 20% is used for testing their performance. A stratified train-test split is applied so that the relative distribution of the different attack classes is approximately preserved in both the training and testing datasets.

6.3 Classification Algorithms

A. Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It calculates the probability of a particular class based on the observed features. Bayes' theorem is represented as:

P(A|B)=P(B|A)P(A)P(B)

For classification, the posterior probability of a class C given the feature vector X is calculated as:

P(C|X)=P(X|C)P(C)P(X)

where C represents the class, X represents the observed features, P(C|X) represents the posterior probability, P(X|C) represents the likelihood, and P(C) represents the prior probability of the class.

Assuming that the features are conditionally independent given the class, the classifier predicts the class using:

C^=arg⁡max⁡CP(C)∏j=1nP(xj|C)

For continuous numerical network features, Gaussian Naive Bayes can be used because it assumes that the continuous features follow a Gaussian distribution within each class.

B. Support Vector Machine

Support Vector Machine (SVM) is a supervised classification algorithm that attempts to identify an optimal decision boundary, or hyperplane**,** that separates different classes in the feature space. The decision function of a linear SVM can be represented as:

f(x)=wTx+b

where w represents the weight vector and b represents the bias term.

The basic optimization objective of SVM is:

$$\min_{w,b} \frac{1}{2}||w||^2$$

subject to the constraint:

$$y_i(w^T x_i + b) \geq 1$$

For complex and nonlinear network traffic patterns, a Radial Basis Function (RBF) kernel can be used. The RBF kernel is defined as:

$$K(x_i, x_j) = e^{-\gamma||x_i - x_j||^2}$$

where γ controls the influence of individual training observations. Since SVM is sensitive to the scale of input features, feature standardization is important before training the model.

C. Decision Tree

A Decision Tree is a supervised classification algorithm that recursively divides the dataset into smaller groups based on feature values. At each node, the algorithm selects a feature and a splitting condition that best separates the different classes. For example, a network-flow feature such as flow duration can be used to divide observations into different groups, which can subsequently be classified as normal or malicious traffic.

One commonly used criterion for determining the quality of a split is Gini impurity, which is calculated as:

$$Gini(S) = 1 - \sum_{i=1}^{C} p_i^2$$

where C represents the number of classes and pi represents the proportion of observations belonging to class i. The Decision Tree attempts to select splits that reduce the impurity of the resulting groups.

D. Random Forest

Random Forest is an ensemble classification algorithm that combines the predictions of multiple Decision Trees. Each tree is trained using a random sample of observations and a random subset of features. The final prediction is generally determined using majority voting among the individual trees and can be represented as:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), ..., T_B(x)\}$$

where T1,T2,…,TB represent the individual Decision Trees and B represents the total number of trees in the forest. Random Forest generally provides better generalization than a single Decision Tree because combining multiple trees reduces variance and makes the model less dependent on a particular training sample.

6.4 Evaluation Metrics

Since the RT-IoT2022 dataset contains multiple attack classes with an unequal number of observations**,** accuracy alone is not sufficient to evaluate the classification models**.** Therefore,

multiple evaluation metrics are used to provide a comprehensive assessment of model performance.

Accuracy measures the proportion of correctly classified observations out of all observations and is calculated as:

Accuracy=TP+TN/TP+TN+FP+FN

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively.

Precision measures the proportion of correctly predicted positive observations among all observations predicted as positive. It is calculated as:

Precision=TP/TP+FP

A higher precision indicates that the model produces fewer false-positive predictions.

Recall, also known as sensitivity, measures the proportion of actual positive observations that are correctly identified by the classifier:

Recall=TP/TP+FN

Recall is particularly important in cybersecurity because a false negative may represent an attack that has not been detected by the intrusion-detection system.

F1-score provides a balance between precision and recall and is calculated as:

F1=2(Precision×Recal)/(Precision+Recall)

A higher F1-score indicates that the classifier provides a good balance between detecting positive instances and minimizing false-positive predictions.

Since RT-IoT2022 is a multiclass and imbalanced dataset **,** Macro-F1 is also considered. Macro-F1 calculates the F1-score independently for each class and then takes the average:

F1macro=1C∑i=1CF1i

where C represents the total number of classes and F1i represents the F1-score of the ith class. Macro-F1 is particularly useful because every attack category receives equal importance, preventing the majority class from dominating the overall evaluation.

A confusion matrix is also generated for each classifier to visualize the relationship between actual and predicted classes. It helps identify correctly classified observations as well as the specific attack categories that are frequently misclassified.

ROC-AUC is another evaluation metric that measures the ability of a classifier to distinguish between different classes across various classification thresholds. For the multiclass problem, a macro-averaged ROC-AUC can be calculated when the model provides appropriate probability estimates.

Finally, training time and prediction time are recorded for each classifier. These computational measurements are useful because an intrusion-detection system may need to process network traffic quickly and operate in a near-real-time environment.

7. Implementation

7.1 Tools and Libraries Used

The implementation of the proposed classification system is carried out using Python as the primary programming language. Jupyter Notebook can be used as the development environment. The Pandas library is used for data loading, manipulation, cleaning, and analysis, while NumPy is used for numerical computations. The Scikit-learn library is used to implement the Naive Bayes, SVM, Decision Tree, and Random Forest algorithms, along with preprocessing and evaluation metrics. Matplotlib and Seaborn are used to create graphs, statistical plots, confusion matrices, and other visualizations. The Ucimlrepo library can be used to retrieve the RT-IoT2022 dataset directly from the UCI Machine Learning Repository.

7.2 Data / Input

The dataset selected for this study is the RT-IoT2022 dataset**,** which is publicly available through the UCI Machine Learning Repository**.** The dataset was developed for research involving intrusion detection in IoT and real-time network environments.

According to the UCI Machine Learning Repository, RT-IoT2022 contains 123,117 observations and 83 usable input features**.** The dataset contains network-flow information with both numerical and categorical attributes and does not contain missing values. It includes both normal network traffic and different types of malicious network traffic, making it suitable for a multiclass intrusion-detection problem.

The attack categories represented in the dataset include DOS_SYN_Hping, ARP_poisoning, NMAP_UDP_SCAN, NMAP_XMAS_TREE_SCAN, NMAP_OS_DETECTION, NMAP_TCP_scan, DDOS_Slowloris, Metasploit_Brute_Force_SSH, and NMAP_FIN_SCAN**,** among other traffic categories.

The dataset also contains normal IoT traffic associated with activities and devices such **as** MQTT, ThingSpeak, Wipro bulb, and Amazon Alexa**.** Therefore, the dataset provides a combination of normal and malicious IoT network traffic that can be used to train and evaluate machine-learning classification models for intrusion detection.

8. Results :

8.1 Model Comparison

The four classification algorithms Naive Bayes, Support Vector Classifier (SVC), Decision Tree, and Random Forest were evaluated on the RT-IoT2022 dataset. The models were compared using accuracy, macro-precision, macro-recall, and macro-F1 score. Macro-averaged metrics were considered particularly important because the dataset contains substantial differences in the number of samples across attack categories.

The comparison table and visualization show the relative performance of the four classifiers. The best-performing model should be determined primarily using macro-F1 and recall, rather than accuracy alone, because a high accuracy can mask poor detection of minority attack classes.

8.2 Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV for SVC, Decision Tree, and Random Forest. For SVC, parameters such as C, gamma, and kernel were considered. For Decision Tree, maximum depth, minimum samples split, and splitting criterion were evaluated. For Random Forest, number of estimators, maximum depth, and minimum samples split were considered.

The final tuned models were then evaluated on the held-out test set. This provides a fairer comparison between the algorithms while maintaining the same test data.

8.3 Confusion Matrix Analysis

The confusion matrices provide class-wise information about correct and incorrect predictions. They help identify which attack categories are detected reliably and which categories are frequently confused with other traffic classes.

For this dataset, confusion-matrix analysis is particularly important because the RT-IoT2022 dataset contains both normal IoT traffic and multiple malicious traffic categories.

8.4 Insights Derived

From the experimental results, the following observations can be made:

Accuracy alone is insufficient for selecting the final classifier because of the imbalanced distribution of attack classes.

Macro-F1 and macro-recall provide a more meaningful comparison of performance across both common and less frequent attack categories.

SVC and tree-based models require different computational resources, with SVC taking considerably longer during hyperparameter optimization on the full dataset.

Decision Tree and Random Forest can model nonlinear relationships between network-flow features without requiring feature scaling.

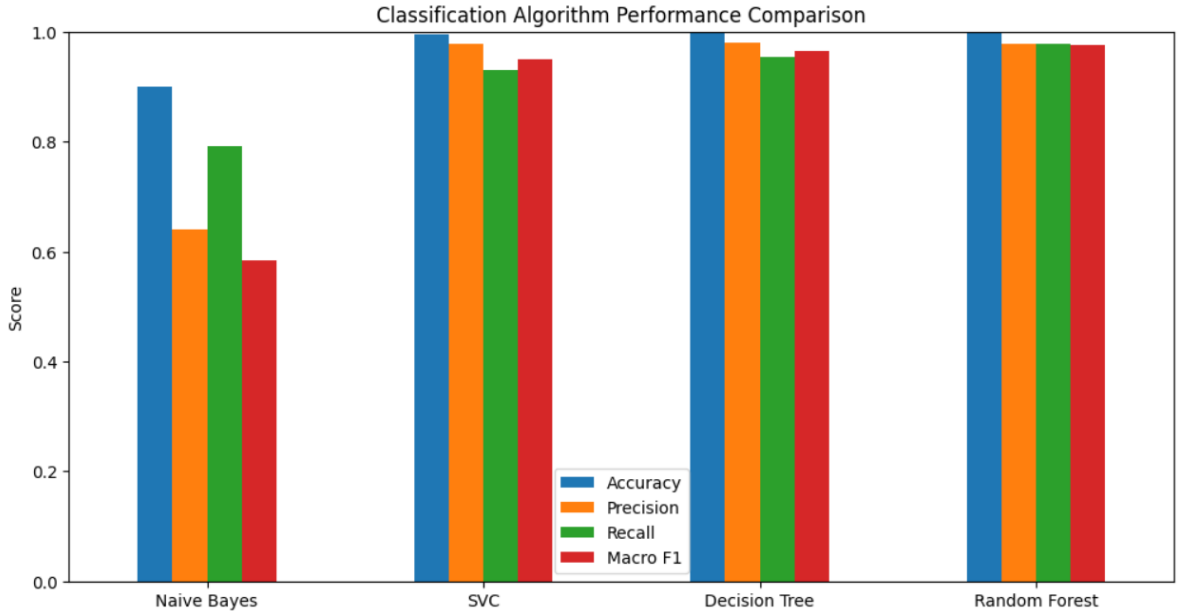
Random Forest provides an ensemble-based approach that can improve robustness compared with an individual Decision Tree.

The confusion matrices provide additional evidence about which individual attack categories are difficult to distinguish.

The final model should therefore be selected by considering classification performance together with minority-class detection and computational cost, rather than simply choosing the model with the highest accuracy. This is also the criterion specified in the assignment.

	Model	Accuracy	Precision	Recall	Macro F1	Training Time (seconds)	Prediction Time (seconds)
0	Naive Bayes	0.900260	0.639366	0.790712	0.583872	0.577601	0.179546
1	SVC	0.995249	0.977288	0.929276	0.950088	20.420759	10.424610
2	Decision Tree	0.997929	0.979975	0.953455	0.965666	1.561309	0.010210
3	Random Forest	0.998457	0.977717	0.977868	0.975967	21.597522	0.496402

Model comparison



Visualization

9. Conclusion

This experiment investigated the application of four classical machine-learning classification algorithms Naive Bayes, Support Vector Machine, Decision Tree, and Random Forest for multiclass IoT intrusion detection using the RT-IoT2022 dataset.

The dataset provides a realistic classification problem containing normal IoT traffic as well as diverse cyberattack patterns. It contains 123,117 observations and 83 usable input features, including both numerical and categorical network-flow attributes.

The data was preprocessed by removing the irrelevant identifier column, encoding categorical attributes using one-hot encoding, splitting the data into training and testing sets using stratification, and applying feature scaling where required. The four classifiers were subsequently trained and evaluated using accuracy, precision, recall, macro-F1 score, classification reports, and confusion matrices.

Hyperparameter optimization was also incorporated for SVC, Decision Tree, and Random Forest to investigate whether suitable parameter configurations could improve classification performance. The results demonstrate that model selection for an imbalanced intrusion-detection problem should not rely solely on overall accuracy. Macro-F1, macro-recall, class-wise confusion patterns, and computational cost should

also be considered when determining the most suitable classifier. This aligns directly with the assignment's stated evaluation objective.

Overall, the experiment demonstrates how classical machine-learning algorithms can be applied to network-flow data for multiclass IoT intrusion detection and provides a basis for further improvements through feature selection, imbalance handling, explainability, advanced models, and real-time deployment.

10. Future Scope:

The following improvements can be explored in future work:

1. Feature Selection: Apply Information Gain, Mutual Information, Recursive Feature Elimination or PCA to reduce the number of network features.
2. Class Imbalance Handling: Apply SMOTE, class weighting or other resampling strategies to improve detection of rare attack classes.
3. Hyperparameter Optimization: Optimize SVM parameters such as C and γ, and optimize Decision Tree/Random Forest parameters such as tree depth and number of estimators.
4. Explainable AI: Use SHAP or permutation importance to identify which network-flow features contribute most to attack detection.
5. Advanced Models: Compare the classical algorithms with XGBoost, LightGBM, neural networks, autoencoders or newer architectures.
6. Real-Time IDS: Develop a lightweight model capable of detecting attacks in real-time IoT environments.
7. Cross-Dataset Validation: Train on RT-IoT2022 and test on another IoT intrusion dataset to investigate how well the models generalize to unseen network environments.

Recent work is already moving toward feature selection, explainability, adaptive inference and advanced neural models on RT-IoT2022, making these particularly relevant future directions.